

GenAI #1: Grok

Accessed through: Grok website

GenAI #2: Microsoft Copilot

Accessed through: Microsoft Copilot website

Prompt used:

Write a C program for this assignment using a MaxHeap made from scratch to prioritize emails for a CEO.

The priorities are Boss, Subordinate, Peer, ImportantPerson, and OtherPerson from highest to lowest. If two emails have the same priority, the newest one should come first. Dates are MM-DD-YYYY.

The commands are EMAIL, NEXT, READ, and COUNT. EMAIL adds an email, NEXT shows the highest priority email without removing it, READ removes it without displaying it, and COUNT shows how many emails are left.

EMAIL format is:

EMAIL <sender category>,<subject>,<date>

For NEXT print:

Next email:

Sender: <sender>

Subject: <subject>

Date: <date>

For COUNT print:

There are <number> emails to read.

Also handle empty emails, two NEXTs in a row, and two READs in a row. The subject can have spaces but no commas.

Don't use a heap library. Make the MaxHeap functions yourself.

Grok generated code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX_EMAILS 10000
```

```
#define MAX_SENDER 50
```

```
#define MAX_SUBJECT 256
```

```
#define MAX_DATE 20
```

```
#define MAX_LINE 512
```

```
typedef struct {
```

```
    char sender[MAX_SENDER];
```

```
    char subject[MAX_SUBJECT];
```

```
    char date[MAX_DATE];
```

```
    int priority; /* higher number = higher priority */
```

```

    int date_val; /* YYYYMMDD for easy comparison; newer = larger */
} Email;

typedef struct {
    Email data[MAX_EMAILS];
    int size;
} MaxHeap;

/* Priority mapping: Boss > Subordinate > Peer > ImportantPerson > OtherPerson */
int get_priority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "OtherPerson") == 0) return 1;
    return 0; /* unknown / fallback */
}

/* Convert MM-DD-YYYY to YYYYMMDD integer */
int parse_date(const char *date_str) {
    int month, day, year;
    if (sscanf(date_str, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0;
    }
    return year * 10000 + month * 100 + day;
}

```

```
/* Return 1 if a has higher priority than b (should be above b in max-heap) */
```

```
int email_greater(const Email *a, const Email *b) {
```

```
    if (a->priority != b->priority) {
```

```
        return a->priority > b->priority;
```

```
    }
```

```
    /* Same priority: newer date first */
```

```
    return a->date_val > b->date_val;
```

```
}
```

```
void swap_email(Email *a, Email *b) {
```

```
    Email tmp = *a;
```

```
    *a = *b;
```

```
    *b = tmp;
```

```
}
```

```
void heap_swim(MaxHeap *h, int idx) {
```

```
    while (idx > 0) {
```

```
        int parent = (idx - 1) / 2;
```

```
        if (email_greater(&h->data[idx], &h->data[parent])) {
```

```
            swap_email(&h->data[idx], &h->data[parent]);
```

```
            idx = parent;
```

```
        } else {
```

```
            break;
```

```
        }
```

```
}
```

```
}
```

```
void heap_sink(MaxHeap *h, int idx) {
```

```
    int n = h->size;
```

```
    while (1) {
```

```
        int left = 2 * idx + 1;
```

```
        int right = 2 * idx + 2;
```

```
        int largest = idx;
```

```
        if (left < n && email_greater(&h->data[left], &h->data[largest])) {
```

```
            largest = left;
```

```
        }
```

```
        if (right < n && email_greater(&h->data[right], &h->data[largest])) {
```

```
            largest = right;
```

```
        }
```

```
        if (largest == idx) break;
```

```
        swap_email(&h->data[idx], &h->data[largest]);
```

```
        idx = largest;
```

```
    }
```

```
}
```

```
void heap_insert(MaxHeap *h, const Email *e) {
```

```
    if (h->size >= MAX_EMAILS) {
```

```
        /* Heap full – ignore or handle as needed; assignment typically assumes enough space
        */
```

```

        return;
    }
    h->data[h->size] = *e;
    heap_swim(h, h->size);
    h->size++;
}

```

/* Peek at the highest-priority email without removing it.

Returns 1 on success, 0 if empty. */

```

int heap_peek(const MaxHeap *h, Email *out) {
    if (h->size == 0) return 0;
    *out = h->data[0];
    return 1;
}

```

/* Remove the highest-priority email.

Returns 1 on success, 0 if empty. */

```

int heap_extract(MaxHeap *h, Email *out) {
    if (h->size == 0) return 0;
    if (out) *out = h->data[0];
    h->size--;
    if (h->size > 0) {
        h->data[0] = h->data[h->size];
        heap_sink(h, 0);
    }
    return 1;
}

```

```
}
```

```
/* Trim leading/trailing whitespace */
```

```
void trim(char *s) {
```

```
    char *start = s;
```

```
    while (*start && isspace((unsigned char)*start)) start++;
```

```
    if (start != s) memmove(s, start, strlen(start) + 1);
```

```
    size_t len = strlen(s);
```

```
    while (len > 0 && isspace((unsigned char)s[len - 1])) {
```

```
        s[--len] = '\0';
```

```
    }
```

```
}
```

```
/* Parse an EMAIL command line.
```

```
Format: EMAIL <sender>,<subject>,<date>
```

```
Subject may contain spaces but no commas.
```

```
Returns 1 on success, 0 on parse failure. */
```

```
int parse_email_line(const char *line, Email *e) {
```

```
    /* Skip the "EMAIL" keyword and any following whitespace */
```

```
    const char *p = line;
```

```
    if (strncmp(p, "EMAIL", 5) != 0) return 0;
```

```
    p += 5;
```

```
    while (*p && isspace((unsigned char)*p)) p++;
```

```
    /* Now p points to sender,subject,date */
```

```
char buf[MAX_LINE];

strncpy(buf, p, sizeof(buf) - 1);
buf[sizeof(buf) - 1] = '\0';

/* Find first comma (end of sender) */
char *c1 = strchr(buf, ',');
if (!c1) return 0;
*c1 = '\0';
char *sender = buf;
trim(sender);

/* Find second comma (end of subject) */
char *c2 = strchr(c1 + 1, ',');
if (!c2) return 0;
*c2 = '\0';
char *subject = c1 + 1;
trim(subject);

char *date = c2 + 1;
trim(date);

if (sender[0] == '\0' || date[0] == '\0') return 0;

strncpy(e->sender, sender, MAX_SENDER - 1);
e->sender[MAX_SENDER - 1] = '\0';
strncpy(e->subject, subject, MAX_SUBJECT - 1);
```

```

e->subject[MAX_SUBJECT - 1] = '\0';
strncpy(e->date, date, MAX_DATE - 1);
e->date[MAX_DATE - 1] = '\0';

e->priority = get_priority(e->sender);
e->date_val = parse_date(e->date);

return 1;
}

int main(void) {
    MaxHeap heap = {0};
    char line[MAX_LINE];

    while (fgets(line, sizeof(line), stdin) != NULL) {
        /* Remove trailing newline */
        size_t len = strlen(line);
        if (len > 0 && line[len - 1] == '\n') {
            line[len - 1] = '\0';
            len--;
        }
        if (len == 0) continue;

        if (strncmp(line, "EMAIL", 5) == 0) {
            Email e;
            if (parse_email_line(line, &e)) {

```

```

        heap_insert(&heap, &e);
    }
}
else if (strcmp(line, "NEXT") == 0) {
    Email top;
    if (heap_peek(&heap, &top)) {
        printf("Next email:\n");
        printf("Sender: %s\n", top.sender);
        printf("Subject: %s\n", top.subject);
        printf("Date: %s\n", top.date);
    } else {
        /* Handle empty heap (also covers consecutive NEXTs on empty) */
        printf("No emails to read.\n");
    }
}
else if (strcmp(line, "READ") == 0) {
    /* Remove highest-priority email without printing.
       Safe on empty heap (two READs in a row, or READ when empty). */
    heap_extract(&heap, NULL);
}
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap.size);
}
/* Unknown commands are ignored */
}

```

```
    return 0;
}
```

Copilot generated code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_EMAILS 1000
```

```
#define MAX_SUBJECT 200
```

```
#define MAX_CATEGORY 30
```

```
typedef struct {
    char sender[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    char date[20];

    int priority;
    int dateValue;
} Email;
```

```
typedef struct {
    Email heap[MAX_EMAILS];
    int size;
} MaxHeap;
```

```
/* Convert sender category to priority value */
```

```

int getPriority(char *sender) {
    if (strcmp(sender, "Boss") == 0)
        return 5;
    if (strcmp(sender, "Subordinate") == 0)
        return 4;
    if (strcmp(sender, "Peer") == 0)
        return 3;
    if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    return 1; /* OtherPerson */
}

```

/* Convert MM-DD-YYYY to integer YYYYMMDD */

```

int dateToInt(char *date) {
    int month, day, year;
    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

```

/* Return 1 if a has higher priority than b */

```

int higherPriority(Email a, Email b) {
    if (a.priority > b.priority)
        return 1;

    if (a.priority < b.priority)

```

```
return 0;
```

```
return a.dateValue > b.dateValue;
```

```
}
```

```
void swap(Email *a, Email *b) {
```

```
    Email temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
void heapifyUp(MaxHeap *h, int index) {
```

```
    while (index > 0) {
```

```
        int parent = (index - 1) / 2;
```

```
        if (higherPriority(h->heap[index], h->heap[parent])) {
```

```
            swap(&h->heap[index], &h->heap[parent]);
```

```
            index = parent;
```

```
        } else {
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
void heapifyDown(MaxHeap *h, int index) {
```

```
    while (1) {
```

```

int left = 2 * index + 1;

int right = 2 * index + 2;

int largest = index;


if (left < h->size &&
    higherPriority(h->heap[left], h->heap[largest])) {
    largest = left;
}


if (right < h->size &&
    higherPriority(h->heap[right], h->heap[largest])) {
    largest = right;
}


if (largest != index) {
    swap(&h->heap[index], &h->heap[largest]);
    index = largest;
} else {
    break;
}

}

}

}


void insertEmail(MaxHeap *h, Email email) {
    if (h->size >= MAX_EMAILS)
        return;

```

```
h->heap[h->size] = email;
heapifyUp(h, h->size);
h->size++;
}
```

```
Email peek(MaxHeap *h) {
return h->heap[0];
}
```

```
void removeMax(MaxHeap *h) {
if (h->size == 0)
return;
```

```
h->heap[0] = h->heap[h->size - 1];
h->size--;
```

```
if (h->size > 0)
heapifyDown(h, 0);
}
```

```
int main() {
MaxHeap inbox;
inbox.size = 0;
```

```
char line[500];
```

```
while (fgets(line, sizeof(line), stdin) != NULL) {
```

```
    line[strcspn(line, "\n")] = '\0';
```

```
    if (strncmp(line, "EMAIL ", 6) == 0) {
```

```
        Email e;
```

```
        char *data = line + 6;
```

```
        char *token = strtok(data, ",");
```

```
        if (token == NULL)
```

```
            continue;
```

```
        strcpy(e.sender, token);
```

```
        token = strtok(NULL, ",");
```

```
        if (token == NULL)
```

```
            continue;
```

```
        strcpy(e.subject, token);
```

```
        token = strtok(NULL, ",");
```

```
        if (token == NULL)
```

```
            continue;
```

```
strcpy(e.date, token);
```

```
e.priority = getPriority(e.sender);
```

```
e.dateValue = dateToInt(e.date);
```

```
insertEmail(&inbox, e);
```

```
}
```

```
else if (strcmp(line, "NEXT") == 0) {
```

```
if (inbox.size == 0) {
```

```
printf("No emails available.\n");
```

```
} else {
```

```
Email e = peek(&inbox);
```

```
printf("Next email:\n");
```

```
printf("Sender: %s\n", e.sender);
```

```
printf("Subject: %s\n", e.subject);
```

```
printf("Date: %s\n", e.date);
```

```
}
```

```
}
```

```
else if (strcmp(line, "READ") == 0) {
```

```
if (inbox.size == 0) {
```

```
printf("No emails available.\n");  
  
} else {  
    removeMax(&inbox);  
}  
  
}  
  
else if (strcmp(line, "COUNT") == 0) {  
  
    printf("There are %d emails to read.\n", inbox.size);  
}  
  
}  
  
return 0;  
}
```

Correctness:

Both Grok and Copilot compiled and worked correctly with the sample test file. I also tested things like using NEXT twice, READ twice, different sender priorities, and emails from the same category with different dates. Both programs handled these tests correctly. One small difference was that Copilot printed a message when READ was used on an empty inbox, while Grok did not print anything.

Execution Time:

Both programs use a MaxHeap, so adding an email takes $O(\log n)$ time. READ also takes $O(\log n)$ because the heap has to be fixed after removing an email. NEXT is $O(1)$ because the highest priority email is already at the top of the heap. COUNT is also $O(1)$.

Space Complexity:

Both programs use $O(n)$ space because they need to store all the emails in the heap. Grok set the maximum to 10,000 emails while Copilot used 1,000.

Maintainability:

Both codes are broken into different functions, which makes them easier to read and change. I thought Grok's code was a little easier to follow because it had more comments and separated the input parsing and heap functions clearly. Copilot's code was shorter and simpler.

Selected Program:

I chose Grok's code because it did more checking when reading the emails and the code was easier to follow. It had separate functions for reading the input and doing the MaxHeap operations, so if something went wrong it would be easier to find and fix. Both codes worked in my tests, but Grok seemed like the better one to build my final code from.

Improvements:

I checked the Grok code with the sample test and some extra cases to make sure the heap was working correctly. I checked that NEXT shows the email without removing it, READ removes it, newer emails come first when they have the same sender category, and the sender priorities are correct. I also added more comments to make the final code easier to understand.